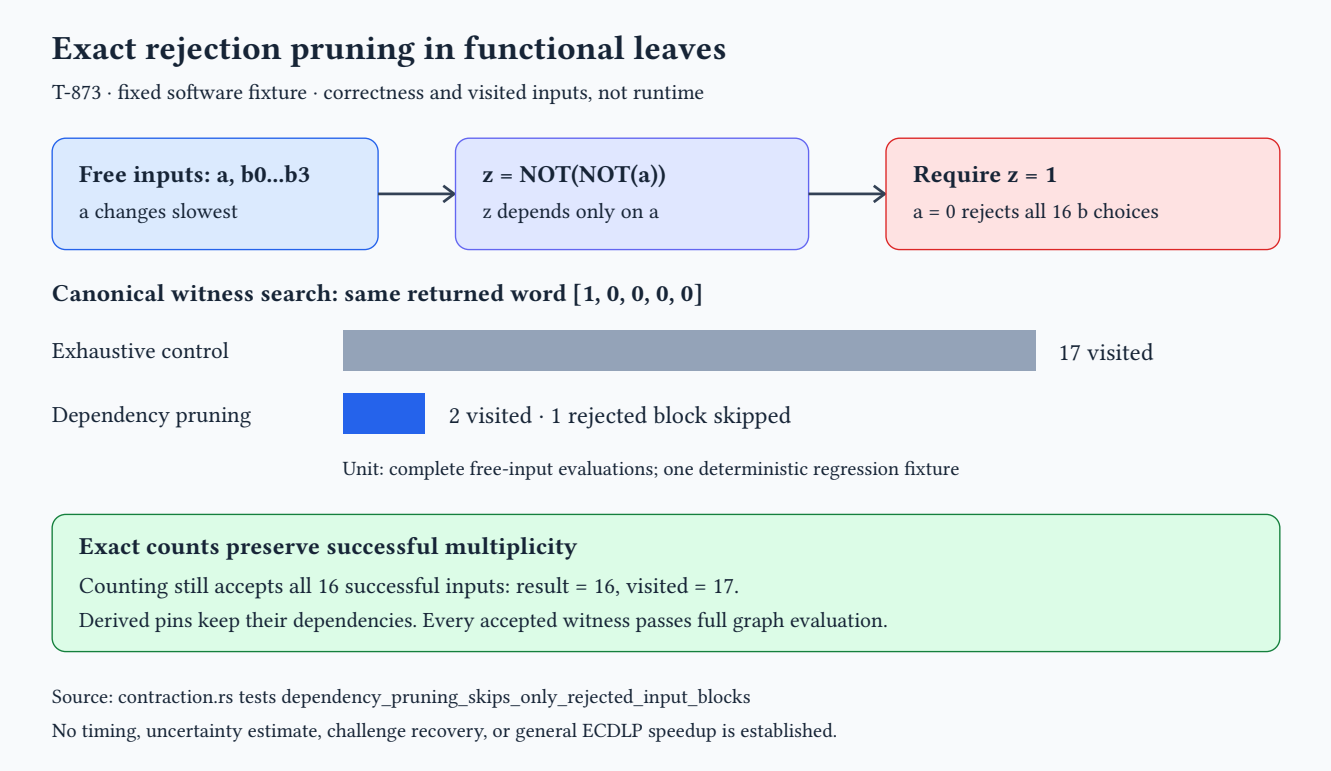
Exact rejection pruning for hashed ECDL

Implementation record · 2026-10-30 UTC · Conductor T-873

Scope. Improve the finite-size functional fallback in the native Boolean tensor / hashed-factor-base implementation. This is software validation; runtime benefit and the general ECDLP objective remain open.



Why the skip preserves the answer

Enumeration digit zero changes fastest. Each signal receives the minimum position of any free input that can affect it. A failing check with minimum position k cannot change while digits below k vary. Resetting those digits and incrementing the suffix skips only rejected inputs. Pinned derived outputs retain the dependencies of their computation. Successful inputs advance one at a time, preserving multiplicity; canonical witness order stays unchanged.

Evidence and controls

The crypto native suite passes 37 tests, including exact full seeded certificate agreement with the exhaustive prefix-count control and independent algebraic replay. New regressions cover 81 interface pin configurations, transitive dependencies, unused inputs, aliases, constant rejection, fully pinned inputs and exhausted budgets. Strict Clippy and Rust 1.78 checks pass. The fixed diagram values are assertions in the native regression, not sampled timing measurements. The Markdown report records mirror and integration outcomes, commands and source hashes.

Boundaries and reproduction

Disable `Counter::functional_pruning` for exhaustive enumeration. Also disable `functional_witnesses` for the original prefix-count control. Preparation is charged to Meter, so tight budgets can be exhausted earlier. There is no measured runtime, large-instance scaling or challenge result. No benchmark or paid cloud job was launched for these checks.

Source base: crypto `bd4c2fe5e6f`. See `docs/hashed-ecdl-functional-plan.md` and the modified `network.rs`, `contraction.rs`, `reduction.rs`. The existing recursive backend and theorem parameters are unchanged.

Primary source: Bochum v0.2, Section 2 and Lemma 1; Section 3 parsimonious encodings. The optimization is standard dependency-based pruning; no novelty claim is made.